

클라우드 인프라 아키텍처 설계서

대용량 이미지 AI 분석 및 생성 서비스를 위한 인프라 구축 방안

문서 종류: 인프라 아키텍처 및 CI/CD 설계서
최종 수정: 2026년 6월 26일
설계 원칙: 보수적 안정성 (Conservative Stability), 고가용성, 결합 격리

1. 개요 및 설계 목적

본 문서의 목적은 로컬 터널링 환경(ngrok, Cloudflare Tunnel 등)에서 검증된 대용량 이미지 처리 및 AI 분석 기반 서비스를 프로덕션 레벨의 클라우드 환경으로 이관하기 위한 아키텍처 설계를 기술하는 데 있다.

해당 서비스는 사용자가 업로드한 원본 이미지를 분석하거나 생성형 AI(OpenAI API 등)를 활용하여 시각적 데이터를 다룬다. 시스템 아키텍처 설계 과정에서 해결해야 할 핵심 과제는 '**대용량 이미지 파일의 빈번한 이동**'과 '**AI API 호출로 인한 대기 타임아웃**' 및 '**서버 메모리 고갈(OOM, Out Of Memory) 위험**'을 제어하는 것이다. 이를 위해 인프라의 **보수적 안정성 (Conservative Stability)**을 최우선 가치로 삼고, 결합 격리 및 비동기 이벤트 기반 아키텍처를 도입한다.

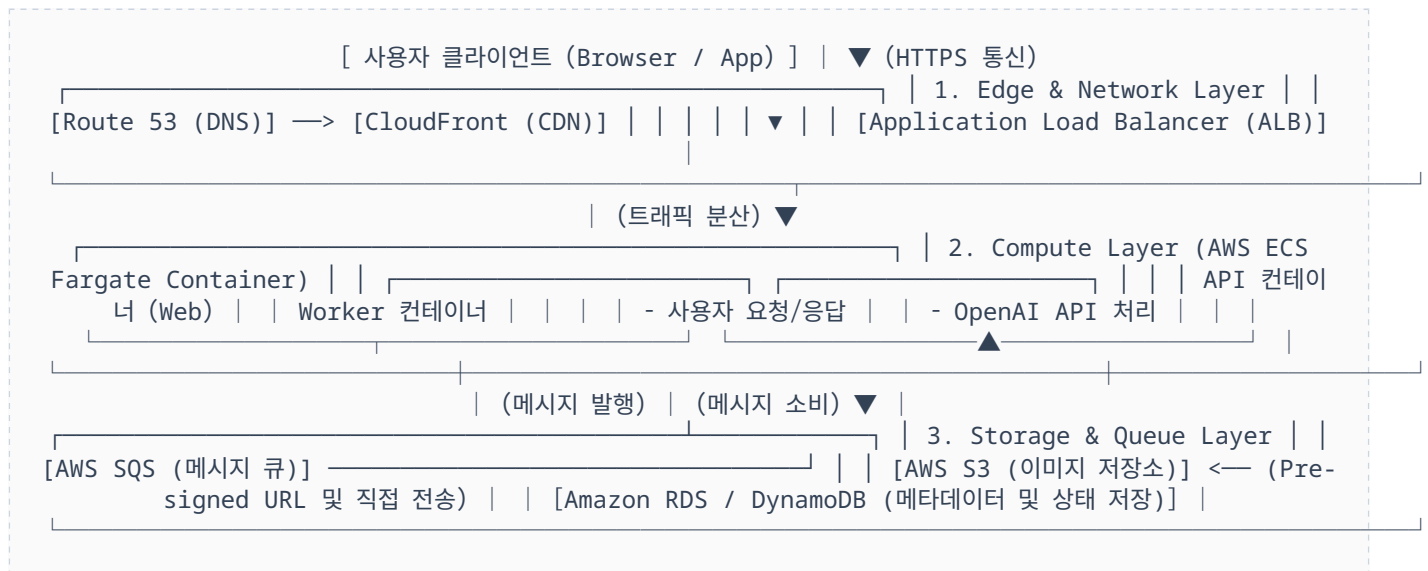
2. 기존 인프라 vs 클라우드 아키텍처 비교

로컬 개발 환경과 프로덕션 클라우드 환경의 핵심 차이점 및 개선 효과는 다음과 같다.

구분	기존 로컬 터널링 (ngrok / Cloudflare)	제안하는 클라우드 아키텍처 (AWS + Docker)
인프라 안정성	로컬 PC 자원에 종속적, 프로세스 다운 시 서비스 중단	AWS Multi-AZ 분산 및 ECS Fargate 활용으로 높은 내결함성 확보
파일 처리 방식	웹 서버 메모리에 이미지 적재 후 API 전송 (OOM 위험 고조)	S3 Pre-signed URL을 이용한 스토리지 직접 업로드 (서버 부하 최소화)
AI 요청 처리	HTTP 동기 요청으로 분석 대기 시간 동안 커넥션 점유 (타임아웃 발생)	AWS SQS 큐를 도입한 비동기 백그라운드 워커 처리 (결합 격리)
배포 및 확장	수동 코드 실행 및 터널 재연결 필요	GitHub Actions 및 AWS ECR/ECS 연동을 통한 무중단 CI/CD 구현

3. AWS 기반 인프라 아키텍처 설계

전체 아키텍처는 결합도를 낮추고 각 컴포넌트의 자원을 독립적으로 확장할 수 있도록 계층형 구조로 설계한다.



3.1. 계층별 상세 컴포넌트 역할

- **Network & Edge Layer:** Route 53을 통해 도메인을 관리하며, CloudFront를 통해 프론트엔드 정적 파일(HTML, JS, CSS)을 전역 캐싱하여 배포한다. ALB는 다수의 API 컨테이너로 상태 확인(Health Check)을 수행하며 가용 상태인 컨테이너에만 동적으로 트래픽을 분산한다.
- **Compute Layer (ECS Fargate):** EC2 인스턴스를 직접 관리하지 않는 서버리스 컨테이너 환경을 채택하여 OS 패치나 인프라 관리 부담을 줄인다. 사용자의 트래픽을 직접 받는 '**API 서비스**'와 시간이 오래 걸리는 AI 태스크를 수행하는 '**Worker 서비스**'로 Task Definition을 분리하여 컨테이너 수준의 격리를 실현한다.
- **Storage & Queue Layer:** AWS SQS는 대용량 트래픽 스파이크 시 버퍼 역할을 수행하여 배후 시스템을 보호한다. S3는 뛰어난 내구성을 자랑하는 객체 스토리지로, 이미지 파일의 영구 저장소 역할을 담당한다.

4. 대용량 이미지 및 AI 병목 최적화 전략

보수적 안정성을 위한 핵심 규칙: 웹 서버 컨테이너는 어떠한 경우에도 대용량 이미지 바이너리 파일을 직접 메모리에 장시간 적재하거나, 외부에 동기식으로 대용량 파일을 전송해서는 안 된다.

4.1. S3 Pre-signed URL을 이용한 파일 직접 업로드 패턴

기존 방식처럼 사용자가 웹 서버로 이미지를 업로드하고 이를 웹 서버가 다시 S3나 외부 API로 보내는 구조는 네트워크 대역폭 허비와 컨테이너 메모리 부족을 유발한다. 이를 해결하는 데이터 흐름은 다음과 같다.

1. 클라이언트가 API 서버에 이미지 업로드 권한을 요청한다.
2. API 서버는 S3 SDK를 사용해 특정 파일 이름에 대해 5분간 유효한 **Pre-signed URL(임시 서명된 URL)**을 생성하여 클라이언트에 즉시 반환한다. (메모리 및 CPU 소모 미비)
3. 클라이언트는 해당 Pre-signed URL을 이용하여 **AWS S3 스토리지로 직접 이미지 파일을 업로드(PUT 요청)**한다.
4. 업로드가 완료되면 클라이언트는 파일의 S3 Key(경로)와 함께 최종 분석 요청을 API 서버로 전송한다.

4.2. SQS 기반 비동기 이벤트 기반 처리 패턴

OpenAI API를 통한 이미지 분석 및 이미지 생성은 모델 크기와 대기열에 따라 수 초에서 수십 초의 시간이 소요된다. HTTP 동기식 연결을 유지할 경우 연결 차단(Connection Blocking) 현상이 발생하므로, SQS를 활용한 비동기 작업 처리를 구축한다.

- **작업 이관:** API 서버는 클라이언트의 분석 요청을 받으면 대기하지 않고, SQS 큐에 메시지(S3 파일 경로, 사용자 ID, 수행할 태스크 정보)를 발행한 뒤 클라이언트에게 "요청 접수 완료 (Status: Processing)" 상태를 즉시 반환한다.
- **느슨한 결합 (Loose Coupling):** Worker 컨테이너는 백그라운드에서 SQS를 롱 폴링(Long Polling)하며 메시지를 하나씩 꺼내 처리한다. OpenAI API 호출 과정에서 타임아웃이 발생하거나 API가 일시적으로 중단되더라도 메시지는 SQS에 보관되므로 데이터 유실 없이 재시도 메커니즘을 수행할 수 있다.
- **결과 반영:** 처리가 완료되면 Worker는 결과를 데이터베이스(RDS/DynamoDB)에 기록하고 상태를 'Complete'로 변경한다. 클라이언트는 웹소켓(WebSocket) 또는 주기적인 폴링을 통해 최종 가공 데이터 및 생성된 이미지 링크를 확인한다.

5. CI/CD 파이프라인 설계 및 지속적 배포

Docker 컨테이너의 장점을 극대화하고 환경 일관성을 유지하기 위해 GitHub Actions를 기반으로 하는 자동화된 무중단 배포 시스템을 구현한다.

5.1. 배포 프로세스 스테이지

1. **Source & 코드 검증:** 개발자가 main 혹은 release 브랜치로 코드를 Push하면 파이프라인이 가동된다. 코드 린팅(Linting) 및 유닛 테스트를 수행하여 무결성을 검증한다.
2. **Docker 이미지 빌드 및 태깅:** 안전성이 검증된 소스코드를 바탕으로 경량 베이스 이미지(예: python:3.11-slim)를 활용해 Docker 이미지를 빌드한다. 이미지 태그에는 커밋 해시(Commit Hash) 또는 버전을 명시하여 추적성을 확보한다.
3. **레지스트리 푸시:** 빌드된 이미지를 안전한 비공개 컨테이너 저장소인 **AWS ECR (Elastic Container Registry)**로 업로드한다.
4. **서비스 업데이트 및 무중단 전환:** GitHub Actions가 AWS ECS에 새 롤링 업데이트(Rolling Update) 배포 명령을 하달한다. ECS는 새 버전의 컨테이너를 먼저 실행하고, ALB의 헬스체크를 통과하면 기존 버전의 컨테이너를 안전하게 드레인(Draining) 후 제거한다. 이로써 서비스 중단 시간(Downtime)이 0이 된다.

5.2. 환경 설정을 위한 Dockerfile 예시 (안정성 확보 기준)

```
# 1단계: 멀티 스테이지 빌드 또는 가벼운 프로덕션 이미지 사용
FROM python:3.11-slim AS base

# 시스템 의존성 최소화 및 보안 패치 적용
RUN apt-get update && apt-get install -y --no-install-recommends \
    curl \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# 가상환경 격리 및 종속성 설치
COPY requirements.txt .
```

```

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Root 권한 실행 방지를 통한 보안 강화
RUN useradd -m appuser && chown -R appuser:appuser /app
USER appuser

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

```

6. 보안 및 모니터링 전략

6.1. 네트워크 및 자원 권한 보안

- **VPC 서브넷 분리:** ALB 및 CloudFront는 외부 인터넷과 통신 가능한 퍼블릭 서브넷(Public Subnet)에 배치하며, 실제 가동되는 ECS 컨테이너 및 데이터베이스(RDS)는 오직 내부 통신만 가능한 프라이빗 서브넷(Private Subnet)에 격리한다. 인터넷 접속이 필요한 대외 API(OpenAI API 등) 호출은 NAT 게이트웨이를 경유하도록 구성한다.
- **IAM 최소 권한 원칙:** API 컨테이너와 Worker 컨테이너에 각각 독립적인 IAM Role을 부여한다. API 컨테이너는 S3 Pre-signed URL 발행 권한과 SQS 메시지 전송(SendMessage) 권한만 가지며, Worker 컨테이너는 SQS 메시지 수신(ReceiveMessage, DeleteMessage) 및 S3 객체 읽기/쓰기 권한만 가지도록 철저히 제어한다.

6.2. 인프라 관측 가능성 (Observability)

- **Amazon CloudWatch 로그 통합:** 모든 컨테이너의 표준 출력(stdout/stderr) 로그를 CloudWatch Logs로 집중화하여 인프라 장애 발생 시 신속히 추적할 수 있도록 돕는다.
- **SQS 백로그 지표 모니터링:** 큐에 쌓여 있는 메시지 수(ApproximateNumberOfMessagesVisible)를 지속적으로 감시한다. 대량의 이미지 분석 요청으로 인해 백로그가 급증할 경우, 알람을 발생시키고 ECS 오토스케일링(Auto Scaling) 정책과 연동하여 Worker 컨테이너 수를 자동으로 동적 확장하도록 설정한다.

7. 결론

본 클라우드 인프라 아키텍처는 초기 로컬 터널링 환경에 비해 복잡성이 증가하는 지점이 있으나, 서비스 운영 환경에서 빈번하게 일어나는 대용량 파일 이동 문제를 완전히 분산형 스토리지(S3 Pre-signed URL) 기반으로 극복하였다. 또한 OpenAI API와 같이 외부 의존성이 높고 무거운 연산을 비동기 메시지 큐(SQS)와 격리된 컨테이너(Worker) 구조로 처리함으로써, 어떠한 트래픽 변동 상황에서도 메인 웹 서비스가 다운되지 않는 강력한 **보수적 안정성**을 달성할 수 있는 인프라 기반을 제공한다.